Лабораторна робота №8
Робота з даними XML та JSON

Крайняк Михайло Юрійович, 451 група

1. Мета і середовище

Мета: навчитися читати, створювати й оновлювати структуровані дані у форматах XML та JSON засобами PHP; використати DOM для XML, масиви для JSON, перевірити помилки розбору й структуру записів та створити каталог із формою додавання за обраним варіантом.

Середовище: XAMPP, **PHP 8.2.12** (x64; та сама версія для вебсервера `php -S` і термінала). Розширення `dom`, `libxml`, `ctype`, `mbstring`, `json` увімкнені. Код сумісний з PHP 8.1.

Варіант 10: розклад маршрутів автовокзалу.
Репозиторій: github.com/mykhailokrainiak/web-progr/tree/main/lab 08

2. Схема полів запису «рейс»

Поле	JSON	XML	Обов'язкове	Допустимі значення
`id`	рядок	текст	так	16 шістнадцяткових символів; генерує сервер через `bin2hex(random_bytes(8))` з перевіркою унікальності
`number`	рядок	текст	так	номер рейсу: 1–10 літер, цифр або дефісів
`destination`	рядок	текст	так	пункт призначення: 2–100 символів (`mb_strlen`), без керівних символів
`departure`	рядок	текст	так	час відправлення `ГГ:ХХ`, 00:00–23:59
`platform`	ціле число	текст із цифр	так	1–20
`price`	ціле число	текст із цифр	так	ціна квитка в гривнях, 1–5000

XML: корінь `<routes>`, записи `<route>`, кожне поле — рівно один дочірній елемент без атрибутів, лише текст. Числа в XML — рядки цифр без початкових нулів, після читання перетворюються на `int`. JSON: масив об'єктів з тими самими ключами; `platform` і `price` мають бути саме числами JSON. Початкові дані — 3 вигадані рейси (Житомир, Біла Церква, Кам'янець-Подільський).

3. Опис файлів

Файл	Походження	Призначення
`src/common.php`, `read_persons_*.php`, `write_persons_*.php`, `persons.xml/json`	Готові приклади	Читання, перевірка й генерація навчальних записів people/name/age
`src/dom_walk.php`	**Власне**	Крок 4: обхід DOM з `documentElement`, `DOMNodeList`, `item()`, `textContent`; виведення таблицею
`src/add_fourth.php`	**Власне**	Крок 7: четвертий запис у копії XML (через DOM) і JSON (через масив), повторне читання й порівняння
`shared/routes.php`	**Власне**	Схема полів, перевірки значень і структури, CSRF, генерація id, блокування циклу «прочитати — змінити — записати», flash-повідомлення
`shared/layout.php`	**Власне**	Шаблон сторінки і форма з підписами, підказками та помилками полів
`shared/catalog_page.php`, `add_page.php`, `edit_page.php`	**Власне**	Перегляд розкладу, додавання, редагування за id (підвищена складність)
`xml/storage.php`, `xml/routes.xml`	**Власне**	XML-версія: розбір DOM із перевіркою структури, збереження всього документа через `save()`
`json/storage.php`, `json/routes.json`	**Власне**	JSON-версія: `json_decode`/`json_encode` з `JSON_THROW_ON_ERROR`, запис з `LOCK_EX`
`xml|json/index.php, add.php, edit.php`	**Власне**	Точки входу: підключають сховище своєї версії і спільну сторінку
`index.php`, `README.md`, `data-initial/`, `tools/checks.sh`	**Власне**	Меню, інструкція запуску, початкові тестові дані, автоматичні перевірки

Обидві версії мають однакові поля, перевірки й операції; відрізняється лише `storage.php` з функціями `loadRoutes()` і `saveRoutes()`. Шлях до файла даних задано константою через `__DIR__`, а не параметром форми.

4. Дослідження прикладів (кроки 2–7)

Кроки 2–3. `read_persons_xml.php` і `read_persons_json.php` показують однакові три записи: Олена (20), Тарас (25), «Клуб <Код> & друзі» (0). У XML вік — текст `<age>20</age>`, у JSON — число `20`; спецсимволи в XML закодовано як `<` і `&`, у JSON вони зберігаються як є.

Крок 4. `load()` читає XML з файла за шляхом, `loadXML()` — з рядка; обидва повертають `bool`. `documentElement` — кореневий елемент `<people>`; `getElementsByTagName()` повертає `DOMNodeList`

— «живий» список вузлів з доступом через `item(i)` (повертає `null`, якщо елемента немає); `textContent` — текст вузла з уже декодованими сутностями. Результат обходу виведено таблицею (рис. 7).

Кроки 5–6. Генератори з термінала: `persons.xml: записано 294 байтів`, `persons.json: записано 205 байтів`; `filesize()` дав ті самі 294 і 205 байтів (байти, а не символи: кирилиця займає по 2 байти). Вміст збігся з оригінальними файлами. Запит генератора через браузер повертає 405. Після повторного читання «Клуб <Код> & друзі» показано звичайним текстом, вік 0 прийнято.

Крок 7. `add_fourth.php` додає запис «Ярина "’" & <Ко>», 19 у копії файлів: XML — `createElement` + `createTextNode` + `appendChild` до `documentElement`, JSON — `$people[] = …` і кодування всього масиву. Результат: `save() = 387 байтів, filesize = 387, записів 4`; JSON — 282 байти, 4 записи; дані XML і JSON після повторного читання однакові.

5. Власний код (ключові фрагменти)

5.1. Перевірка значень запису (shared/routes.php)

```
/**
 * Перевіряє значення одного запису з уже правильними типами.
 * Повертає масив «поле => повідомлення»; порожній масив — запис коректний.
 */
function routeFieldErrors(array $route): array
{
    $errors = [];
    $number = $route['number'] ?? null;
    if (!is_string($number) || preg_match('/\A[0-9A-Za-zA-ЩЬЮЯҐЄІЇа-щьюяґєії-]{1,10}\z/u', $number) !==
1) {
        $errors['number'] = 'Номер рейсу: 1–10 літер, цифр або дефісів.';
    }
    $destination = $route['destination'] ?? null;
    if (!is_string($destination) || trim($destination) !== $destination
        || mb_strlen($destination, 'UTF-8') < 2 || mb_strlen($destination, 'UTF-8') > 100
        || preg_match('/\p{Cc}/u', $destination) === 1) {
        $errors['destination'] = 'Пункт призначення: від 2 до 100 символів, без керівних символів.';
    }
    $departure = $route['departure'] ?? null;
    if (!is_string($departure) || preg_match('/\A([01]\d|2[0-3]):[0-5]\d\z/', $departure) !== 1) {
        $errors['departure'] = 'Час відправлення у форматі ГГ:ХХ (00:00–23:59).';
    }
    $platform = $route['platform'] ?? null;
    if (!is_int($platform) || $platform < PLATFORM_MIN || $platform > PLATFORM_MAX) {
        $errors['platform'] = sprintf('Платформа — ціле число %d–%d.', PLATFORM_MIN, PLATFORM_MAX);
    }
    $price = $route['price'] ?? null;
    if (!is_int($price) || $price < PRICE_MIN || $price > PRICE_MAX) {
        $errors['price'] = sprintf('Ціна — ціле число гривень %d–%d.', PRICE_MIN, PRICE_MAX);
    }
    return $errors;
}
```

5.2. Розбір XML і перевірка структури (xml/storage.php)

```
/** Перетворює DOM-дерево на список записів із перевіркою структури. */
function routesFromXml(DOMDocument $dom): array
{
    $root = $dom->documentElement;
    if ($root === null || $root->tagName !== 'routes' || $root->attributes->length !== 0) {
        throw new RuntimeException('Очікується кореневий елемент routes без атрибутів.');
    }
    $routes = [];
    foreach ($root->childNodes as $node) {
        if ($node instanceof DOMText && trim($node->textContent) === '') {
            continue;
        }
        if (!$node instanceof DOMElement || $node->tagName !== 'route' || $node->attributes->length !==
0) {
            throw new RuntimeException('У routes дозволено лише елементи route без атрибутів.');
        }
        $fields = [];
        foreach ($node->childNodes as $field) {
            if ($field instanceof DOMText && trim($field->textContent) === '') {
                continue;
            }
            if (!$field instanceof DOMElement || !in_array($field->tagName, ROUTE_FIELDS, true)
                || array_key_exists($field->tagName, $fields) || $field->attributes->length !== 0) {
                throw new RuntimeException('Поля route: ' . implode(', ', ROUTE_FIELDS) . ' без повторів
і атрибутів.');
            }
            foreach ($field->childNodes as $part) {
                if (!$part instanceof DOMText) {
                    throw new RuntimeException('Поле ' . $field->tagName . ' має містити лише текст.');
                }
            }
            $fields[$field->tagName] = $field->textContent;
        }
        // XML зберігає текст: числові поля перетворюємо після перевірки формату.
        foreach (['platform', 'price'] as $numeric) {
            if (isset($fields[$numeric])) {
                $fields[$numeric] = strictInt($fields[$numeric]) ?? $fields[$numeric];
            }
        }
        $ordered = [];
        foreach (ROUTE_FIELDS as $name) {
            if (array_key_exists($name, $fields)) {
                $ordered[$name] = $fields[$name];
            }
        }
        $routes[] = $ordered;
    }
    return validateRoutes($routes);
}
```

5.3. Створення XML-документа текстовими вузлами (xml/storage.php)

```
function routesToXml(array $routes): DOMDocument
{
    $dom = new DOMDocument('1.0', 'UTF-8');
    $dom->formatOutput = true;
    $root = $dom->createElement('routes');
    $dom->appendChild($root);
    foreach (validateRoutes($routes) as $route) {
        $node = $dom->createElement('route');
        // Імена елементів — лише з ROUTE_FIELDS, значення — текстовими вузлами.
        foreach (ROUTE_FIELDS as $name) {
            $field = $dom->createElement($name);
```

```
            $field->appendChild($dom->createTextNode((string) $route[$name]));
            $node->appendChild($field);
        }
        $root->appendChild($node);
    }
    return $dom;
}

/** Зберігає весь документ; повертає кількість записаних байтів. */
function saveRoutes(array $routes): int
{
    if (!is_file(STORAGE_FILE) || !is_writable(STORAGE_FILE)) {
        throw new RuntimeException('Файл даних недоступний для запису. Зміни не збережено.');
    }
    $bytes = routesToXml($routes)->save(STORAGE_FILE, LIBXML_NOEMPTYTAG);
    if ($bytes === false) {
        throw new RuntimeException('Не вдалося записати XML. Зміни не збережено.');
    }
    return $bytes;
}
```

5.4. Читання й запис JSON (json/storage.php)

```
function loadRoutes(): array
{
    $json = readText(STORAGE_FILE);
    try {
        // Спочатку відрізняємо масив [] від об’єкта {} — у вигляді масивів PHP вони однакові.
        if (!is_array(json_decode($json, false, 512, JSON_THROW_ON_ERROR))) {
            throw new RuntimeException('Кореневе значення JSON має бути масивом.');
        }
        return validateRoutes(json_decode($json, true, 512, JSON_THROW_ON_ERROR));
    } catch (JsonException) {
        throw new RuntimeException('Некоректний синтаксис JSON.');
    }
}

/** Зберігає весь масив; повертає кількість записаних байтів. */
function saveRoutes(array $routes): int
{
    if (!is_file(STORAGE_FILE) || !is_writable(STORAGE_FILE)) {
        throw new RuntimeException('Файл даних недоступний для запису. Зміни не збережено.');
    }
    try {
        $json = json_encode(validateRoutes(array_values($routes)),
            JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES | JSON_PRETTY_PRINT | JSON_THROW_ON_ERROR);
    } catch (JsonException) {
        throw new RuntimeException('Не вдалося закодувати JSON. Зміни не збережено.');
    }
    $content = $json . "\n";
    $bytes = file_put_contents(STORAGE_FILE, $content, LOCK_EX);
    if ($bytes === false || $bytes !== strlen($content)) {
        throw new RuntimeException('Не вдалося повністю записати JSON. Зміни не збережено.');
    }
    return $bytes;
}
```

5.5. Обробник додавання: POST, CSRF, перевірка, блокування, 303 (shared/add_page.php)

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    try {
        [$route, $errors, $raw] = routeFromInput($_POST);
        checkCsrf();
```

```
        if ($errors !== []) {
            http_response_code(422);
            $formError = 'Запис не додано: перевірте виділені поля.';
        } else {
            withLock(STORAGE_LOCK, function () use ($route): void {
                $routes = loadRoutes(); // Пошкоджений файл не замінюємо — буде виняток.
                $routes[] = ['id' => newRouteId($routes)] + $route;
                saveRoutes($routes);
            });
            // Успіх показуємо лише після збереження, на сторінці каталогу.
            flash('Рейс ' . $route['number'] . ' → ' . $route['destination'] . ' додано.');
            header('Location: index.php', true, 303);
            exit;
        }
    } catch (RuntimeException $exception) {
        http_response_code($exception->getCode() ?: 500);
        $formError = $exception->getMessage();
    }
} elseif ($_SERVER['REQUEST_METHOD'] !== 'GET') {
```

5.6. Редагування: невідомий id не створює запис (shared/edit_page.php)

```
            withLock(STORAGE_LOCK, function () use ($id, $route): void {
                $routes = loadRoutes();
                $index = findRouteIndex($routes, $id);
                if ($index === null) {
                    // Невідомий id не створює новий запис.
                    throw new RuntimeException('Рейс із таким id не знайдено; новий запис не створено.',
404);
                }
                $routes[$index] = ['id' => $id] + $route;
                saveRoutes($routes);
            });
```

6. Знімки екрана

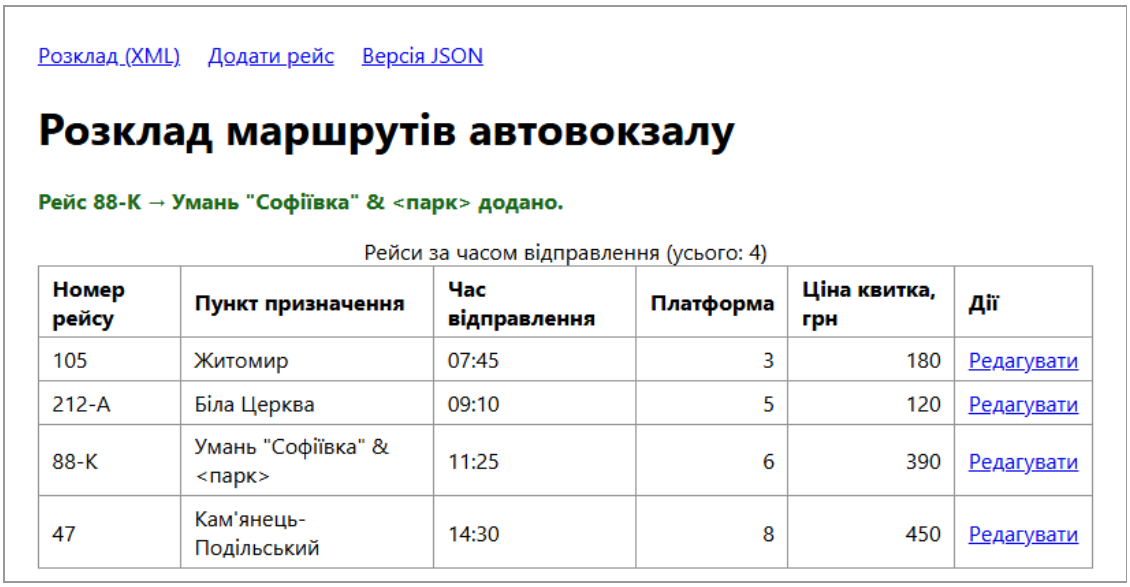
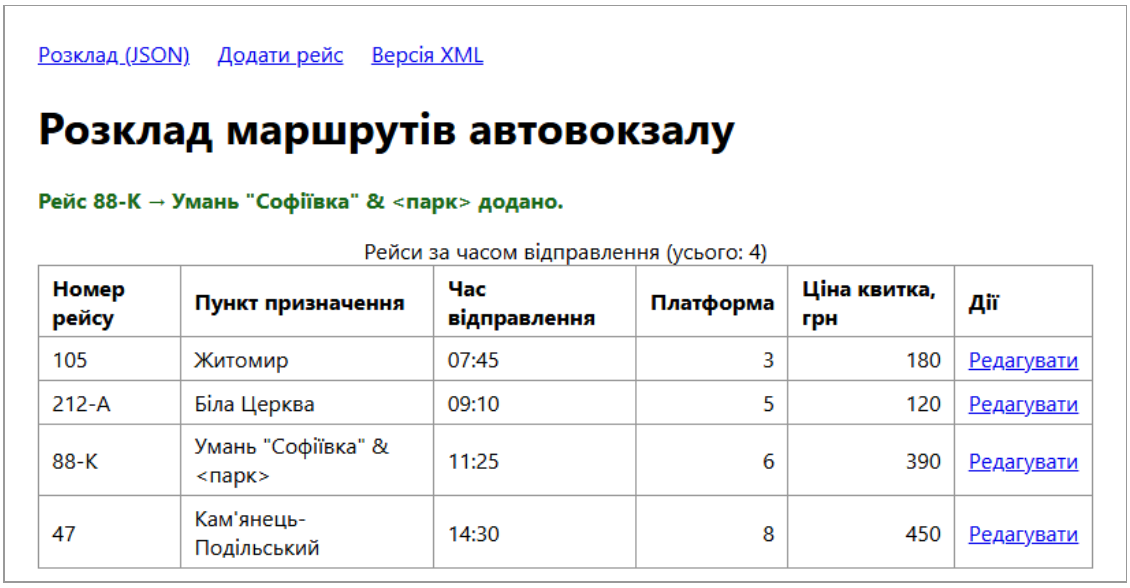


Рис. 1. XML-версія: розклад після додавання рейсу зі спецсимволами (повідомлення показано після перенаправлення 303)



Розклад (JSON) Додати рейс Версія XML

Розклад маршрутів автовокзалу

Рейс 88-К → Умань "Софіївка" & <парк> додано.

Рейси за часом відправлення (усього: 4)

Номер рейсу	Пункт призначення	Час відправлення	Платформа	Ціна квитка, грн	Дії
105	Житомир	07:45	3	180	Редагувати
212-А	Біла Церква	09:10	5	120	Редагувати
88-К	Умань "Софіївка" & <парк>	11:25	6	390	Редагувати
47	Кам'янець-Подільський	14:30	8	450	Редагувати

Рис. 2. JSON-версія: той самий набір даних



Розклад (XML) Додати рейс Версія JSON

Додати рейс

Запис не додано: перевірте виділені поля.

Номер рейсу
105-А
До 10 літер, цифр або дефісів.
Заповніть поле.

Пункт призначення
Л
Від 2 до 100 символів.
Пункт призначення: від 2 до 100 символів, без керівних символів.

Час відправлення
25:00
ГГ:ХХ, наприклад 07:45.
Час відправлення у форматі ГГ:ХХ (00:00–23:59).

Платформа
0
Ціле число 1–20.
Платформа — ціле число 1–20.

Ціна квитка, грн
abc
Ціле число гривень 1–5000.
Ціна — ціле число гривень 1–5000.

Рис. 3. Форма додавання: помилки полів, введення збережено

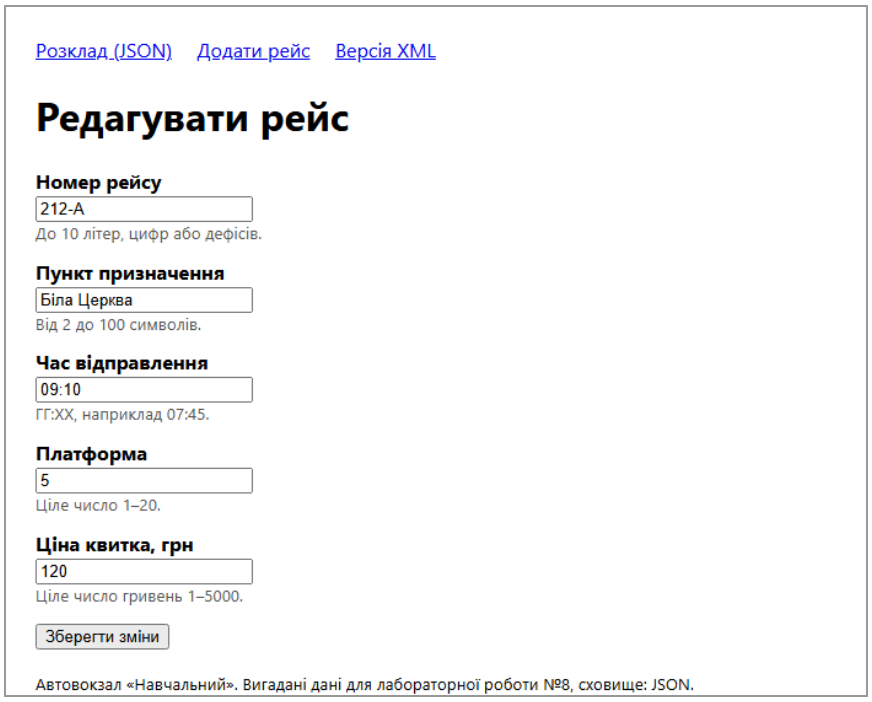


Рис. 4. Редагування рейсу 212-А за id (JSON-версія)



Рис. 5. Редагування невідомого id — відмова (404), форма не показується

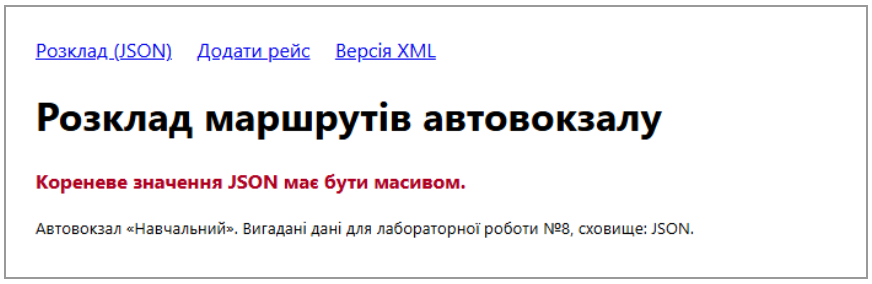
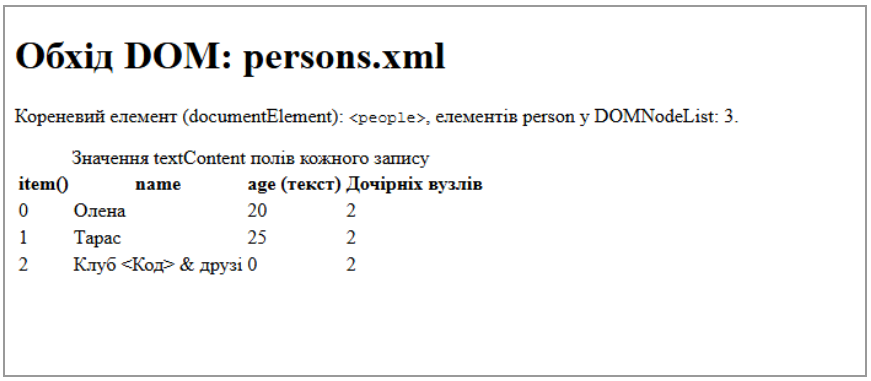


Рис. 6а. JSON-файл містить {} — відмова за структурою



Рис. 6б. XML із незакритим тегом — контрольована відмова в розборі



Обхід DOM: persons.xml

Кореневий елемент (documentElement): <people>, елементів person у DOMNodeList: 3.

Значення textContent полів кожного запису

item()	name	age (текст)	Дочірніх вузлів
0	Олена	20	2
1	Тарас	25	2
2	Клуб <Код> & друзі	0	2

Рис. 7. Обхід DOM persons.xml (крок 4)

Фрагменти файлів після додавання рейсу «88-К» (спецсимволи в XML закодовано, у JSON збережено як є):

```
routes.xml:
  <route>
    <id>ad1114a31070f762</id>
    <number>88-K</number>
    <destination>Умань "Софіївка" &amp;amp; &amp;lt;парк&amp;gt;</destination>
    <departure>11:25</departure>
    <platform>6</platform>
    <price>390</price>
  </route>
```

```
routes.json:
{
    "id": "a508d1dd86ff4986",
    "number": "88-K",
    "destination": "Умань \"Софіївка\" & <парк>",
    "departure": "11:25",
    "platform": 6,
    "price": 390
}
```

7. Результати перевірок

Перевірки виконано скриптом `tools/checks.sh` (curl) для обох версій; повний вивід — `tools/checks-output.txt`. Сторінки перевірено W3C Nu Validator.

Сценарій	Очікувано	Фактично (XML і JSON)	
Початкові приклади XML і JSON	Однакові три записи	Олена, Тарас, «Клуб <Код> & друзі» в обох таблицях	✓
Назва з &, <, лапками; вік 0 (приклад)	Без спотворення, теги не виконуються	У HTML `<b>`, `"`; у XML `<b>&`; у JSON як є; вік 0 прийнято	✓
Порожній `<routes/>` / JSON `[]`	«Записів немає»	200, «Записів немає»; додавання в порожній список працює	✓

Сценарій	**Очікувано**	**Фактично (XML і JSON)**	
Відсутній або порожній файл	Помилка без перезаписування	500, «Файл даних відсутній…» / «Файл даних порожній»; POST — 500, файл не створено й не змінено	✓
Незакритий XML-тег; зайва кома в JSON	Контрольована відмова	«Некоректний синтаксис XML.» / «Некоректний синтаксис JSON.»; файл не змінено	✓
XML із DTD і зовнішньою сутністю	Відмова, сутності не підставляються	«DTD у форматі каталогу не дозволено.»; вміст win.ini не прочитано	✓
Інший корінь XML; JSON `{}` або `null`	Відмова за структурою	«Очікується кореневий елемент routes…» / «Кореневе значення JSON має бути масивом.»	✓
Відсутнє поле; повторне XML-поле; неправильний тип JSON-поля; зайве поле	Відмова до показу таблиці або запису	«Запис 1: відсутнє поле price.», «…без повторів…», `"platform": "1"` і `99.5` відхилено, «містить зайві поля»	✓
Вік -1, 121, 1.5, рядок «20» (приклад)	Відмова	Усі чотири значення відхилено `validatePeople()`	✓
Межі власного каталогу: платформа 0, 21, 1.5, 08; ціна 0, 5001; час 24:00; номер «12 34»; у файлі XML `abc`, `0100`	Відмова	422 з повідомленням біля поля; файл не змінено; у файлі — 500 з номером запису	✓
Додавання двох записів поспіль	Обидва збережено	Рядків 4 → 6, попередні записи на місці	✓
Порожнє поле; масив `number[]` замість рядка	Помилка поля без warning/TypeError	422 «Заповніть поле» / «Поле має бути текстовим значенням»; PHP-повідомлень у відповіді немає	✓
Відсутній / неправильний CSRF-токен	Відмова без зміни файла	403 «Форма застаріла…», контрольна сума файла та сама	✓
Оновлення сторінки після додавання	Повторно не додається	POST → 303 на `index.php`; повторні GET не змінюють кількість (6/6)	✓
Файл недоступний для запису (атрибут «лише читання»)	Повідомлення про збій без успіху	500 «Файл даних недоступний для запису. Зміни не збережено.»; успіх не показано	✓
Редагування невідомого id	Відмова без створення запису	GET і POST — 404; кількість записів і файл без змін; `id=../x` — 400	✓
Редагування існуючого id	Зміни збережено	303, запис оновлено, кількість записів не змінилась	✓
Дані XML- і JSON-версій	Однакові дані	Однаковий результат `loadRoutes()` (md5 серіалізованих масивів збігається)	✓
W3C Markup Validation	Без помилок	0 помилок і попереджень: меню, 3 сторінки прикладів, по 5 станів кожної версії (каталог, форма, форма з помилками, редагування, невідомий id)	✓

8. Відповіді на контрольні питання

1. **XML** — текстова мова розмітки для структурованих даних. Правильно сформований документ має рівно один кореневий елемент, кожен відкритий тег закрито, вкладеність правильна, імена чутливі до регістру, значення атрибутів у лапках, спецсимволи тексту закодовано (`&`, `<`).
2. **JSON** підтримує рядки, числа, `true`, `false`, `null`, об’єкти й масиви. Число `20` декодується в `int`, рядок `"20"` — у `string`; для строгої перевірки (`is_int`) це різні значення.
3. **DOMDocument** подає XML як дерево вузлів, яке можна обходити, перевіряти й змінювати, а потім зберегти цілим. Елемент — вузол із тегом (`<name>`), може мати дочірні вузли й атрибути; текстовий вузол містить лише текст усередині елемента.
4. `load()` і `loadXML()` повертають `bool` (успіх розбору файла / рядка); `save()` — кількість записаних байтів або `false`; `saveXML()` — XML-рядок або `false`.
5. `loadXML()` — метод екземпляра: він заповнює конкретний об’єкт документа, тому його викликають через `$dom->loadXML()`; статичний виклик у PHP 8 — помилка. `item(0)` повертає `null`, якщо елемента немає, і звернення до `->textContent` дало б помилку.
6. Створити елемент `createElement('name')` і додати значення через `createTextNode('A & B')` — DOM сам закодує `&` як `&`. Передавати такий текст другим аргументом `createElement()` не варто: `&` трактується як початок сутності.
7. `json_encode()` перетворює масив PHP на JSON-рядок, `json_decode()` — навпаки (з `true` — в асоціативні масиви). `JSON_THROW_ON_ERROR` кидає `JsonException` замість тихого повернення `null`, тож помилку не сплутати з коректним `null`.
8. Розбір перевіряє лише синтаксис. Синтаксично правильний документ може мати інший корінь, пропущені або повторні поля, рядок замість числа, значення поза межами — тому структуру й типи перевіряють окремо (`routesFromXml`, `validateRoutes`).
9. XML має один корінь: текст після `</routes>` робить документ неправильним. JSON-масив закривається `]`: дописаний об’єкт опиниться поза ним. Тому читають дані, змінюють дерево/масив і записують документ повністю.
10. Екранування XML (`createTextNode`) захищає структуру XML-файла, але після читання `textContent` повертає вже декодований текст з `<` і `&`. Під час виведення в HTML його треба знову екранувати `htmlspecialchars()`, інакше теги виконаються (XSS).
11. Дані змінює лише POST: GET має бути безпечним, бо його виконують посилання, закладки, попереднє завантаження й пошукові роботи. CSRF-токен із сесії підтверджує, що форму надіслано з нашої сторінки. Перенаправлення 303 після успішного запису (PRG) переводить браузер на GET, тож оновлення сторінки не додає запис удруге.
12. Зовнішні сутності (XXE) дозволяють підставити вміст локальних файлів чи URL у документ, виконати запити від імені сервера або перевантажити його (рекурсивні сутності). Тому DTD відхиляємо й не вмикаємо `LIBXML_NOENT`/`LIBXML_DTDLOAD`. Одночасне

оновлення: два запити читають той самий стан і останній запис затирає зміни першого; `LOCK_EX` захищає лише сам запис, тому в роботі весь цикл виконується під `flock` окремого файла `routes.lock`.

9. Висновок

У роботі досліджено читання й запис XML через DOM та JSON через масиви PHP. За варіантом 10 створено дві версії каталогу «Розклад маршрутів автовокзалу» з переглядом, додаванням і редагуванням за id. Вхідні дані перевіряються на сервері (типи, межі, `mb_strlen`), файли — на синтаксис, корінь, набір полів і типи. Помилки не маскуються успішним повідомленням, пошкоджені файли не перезаписуються, а український текст і спецсимволи проходять цикл «запис — читання» без втрат. Усі сценарії перевірки виконано для обох версій, HTML пройшов валідацію W3C без помилок.